Qualité algorithmique (R5.A.08)

Kotlin, les bases du langage

Cédric Lhoussaine

Université de Lille / IUT de Lille

R5.A.08

- **Organisation**
  1. 6 cours (1h)
  2. 6 TP (3h)
  3. CTP (2h)

- **Organisation**
  1. 6 cours (1h)
  2. 6 TP (3h)
  3. CTP (2h)

- **Programme**

  Programmation Kotlin, généricité et polymorphisme, programmation fonctionnelle.

- **Organisation**
  1. 6 cours (1h)
  2. 6 TP (3h)
  3. CTP (2h)

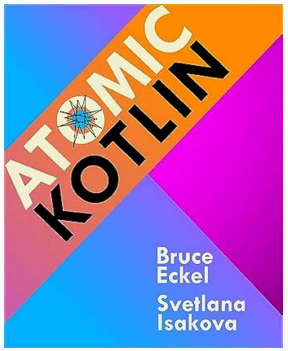
- **Programme**

  Programmation Kotlin, généricité et polymorphisme, programmation fonctionnelle.

- **Bibliographie**

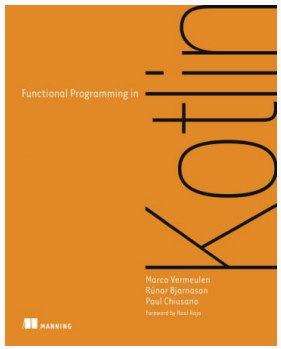
Atomic Kotlin

(B. Eckel and S. Isakova)



Functional Programming in Kotlin

(M. Vermeulen, R. Bjarnason, and P. Chiusano)



Méthodologie de travail: recommandations

Fournitures pour chaque CM et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Méthodologie de travail: recommandations

Fournitures pour chaque CM et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Rentabiliser au maximum les amphis:

- *prendre des notes* (d'où le stylo et les feuilles...)
- poser des questions
- → facilitent la mémorisation et économisent le travail à la maison!

Méthodologie de travail: recommandations

Fournitures pour chaque CM et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Rentabiliser au maximum les amphis:

- *prendre des notes* (d'où le stylo et les feuilles...)
- poser des questions
- → facilitent la mémorisation et économisent le travail à la maison!

Pour chaque CM et TP:

- revoir le(s) cours précédents
- finir les TPs

Méthodologie de travail: recommandations

Fournitures pour chaque CM et TP:

- un stylo (ou plusieurs)
- des feuilles (ou cahier, ...)

Rentabiliser au maximum les amphis:

- *prendre des notes* (d'où le stylo et les feuilles...)
- poser des questions
- → facilitent la mémorisation et économisent le travail à la maison!

Pour chaque CM et TP:

- revoir le(s) cours précédents
- finir les TPs

Tenir à distance tout objet de distraction massive (smartphone, voisin bavard, etc.)

Aujourd'hui

- Présentation de Kotlin.
- Déclaration de variables, typage, structures de contrôle, fonctions, collections…
- Identifier les différences majeures avec Java (syntaxe, concision, null-safety).

Kotlin : histoire et philosophie

- Créé par **JetBrains** (2011), open source
- Objectif : **“less boiler-plate, higher safety”** – code plus lisible, moins d’erreurs.
- Interopérabilité totale avec le byte-code Java; migration progressive possible.
- Multiplateforme : JVM, Android, JavaScript, Native.

« Kotlin brings functional features as first-class citizens while staying 100 % compatible with existing Java libraries »

« Hello World » en Kotlin

```kotlin
fun main(args: Array<String>) {
    println("Hello world !!!!")
}
```

(Remarque: pas de ;)

« Hello World » en Kotlin

```
fun main(args: Array<String>) {
    println("Hello world !!!!")
}
```

(Remarque: pas de ;)

ou bien..

```
fun main() {
    println("Hello world !!!!")
}
```

« Hello World » en Kotlin

```kotlin
fun main(args: Array<String>) {
    println("Hello world !!!!")
}
```

(Remarque: pas de `;`)

ou encore...

```kotlin
fun main() = println("Hello, world!")
```

ou bien..

```kotlin
fun main() {
    println("Hello world !!!!")
}
```

- `fun`: mot-clé fonction.
- Pas de `public static void` → moins de verbosité.
- Le compilateur infère le type `Unit` (équivalent de `void` en C).

Variables et Types

Déclaration de variables et valeurs

```
val monNom : String = "Qualité de développement"
var monAge : Int = 42
```

En Kotlin, on manipule deux sortes de variables :

- variables classiques « mutables » → `var`
- variables « non mutables » (non-modifiables après initialisation) → `val`
- les types peuvent être inférés

Déclaration de variables et valeurs

```
val b = 2   // type inféré
val c : Int // type ne peut être inféré
c = 3       // unique assignation de c car non mutable!
c = 4       // Error: Val cannot be reassigned
```

Il y a aussi des constantes dont la valeur doit être déterminée à la compilation

```
const val NOM = "Robert"            // peut-être déterminé à la compilation
val nomMinuscule = NOM.lowercase() // ne peut pas être une constante
```

Interpolation des chaînes de caractères

- `$x` permet de substituer la variable `x` par sa valeur:

```
fun main () {
    val premiereLettre = 'a'
    val nbLettres = 5
    println("Le mot 'alpha' commence par un $premiereLettre et
                contient $nbLettres lettres")
}
```

Interpolation des chaînes de caractères

- `${...}` substitue l'expression en `{}` par le résultat de son évaluation:

```
fun main () {
    val nbConsonnes = 3
    val nbVoyelles = 2
    println("Le mot 'alpha' contient ${nbConsonnes + nbVoyelles} lettres")
}
```

Types

Entiers			
Type	**Tailles**	**Min**	**Max**
`Byte`	8	-128	127
`Short`	16	-32768	32767
`Int`	32	-2^{31}	$2^{31}-1$
`Long`	64	-2^{63}	$2^{63}-1$

Type	**Tailles**
Flottants	
`Float`	32
`Double`	64
Caractères	
`Char`	16
`String`	variable
Booléens	
`Boolean`	8

Conversions de types

Par défaut, les nombre décimaux sont interprétés comme `Double`:

```
val pi = 3.14 // type inféré : Double
val pifloat = 3.14f // type inféré : Float
```

Conversions de types

Par défaut, les nombre décimaux sont interprétés comme `Double`:

```
val pi = 3.14 // type inféré : Double
val pifloat = 3.14f // type inféré : Float
```

⚠️ Il n'y a pas de conversion implicite entre les nombres:

```
fun foo(x : Double) : Double {
    return x
}
```

Conversions de types

Par défaut, les nombre décimaux sont interprétés comme `Double`:

```
val pi = 3.14 // type inféré : Double
val pifloat = 3.14f // type inféré : Float
```

⚠️ Il n'y a pas de conversion implicite entre les nombres:

```
fun foo(x : Double) : Double {
    return x
}
```

```
print(foo(1)) //Error: The integer
literal does not conform to the
expected type Double
```

Conversions de types

Par défaut, les nombre décimaux sont interprétés comme `Double`:

```
val pi = 3.14 // type inféré : Double
val pifloat = 3.14f // type inféré : Float
```

⚠️ Il n'y a pas de conversion implicite entre les nombres:

```
fun foo(x : Double) : Double {
    return x
}
```

```
print(foo(1)) //Error: The integer
literal does not conform to the
expected type Double
```

🔓 Solution: conversion explicite

```
print(foo(1.toDouble()))
```

Null safety

- Objectif de Kotlin: éliminer (le plus possible) des **erreurs de références nulles** (`NullPointerException`)
- le système de type de Kotlin distingue les références qui peuvent être `null` de celles qui ne le peuvent pas

Null safety

- Objectif de Kotlin: éliminer (le plus possible) des **erreurs de références nulles** (`NullPointerException`)
- le système de type de Kotlin distingue les références qui peuvent être `null` de celles qui ne le peuvent pas
- par défaut les types n'autorisent pas les valeurs `null`:

```
var aSafe : String = "Kotlin"
aSafe = null
```

→ **`Null can not be a value of a non-null type String`**

- ce qui permet d'accéder à une propriété d'une variable de façon sûre:

```
println(aSafe.length)
```

→ 6

Null safety

- pour autoriser les valeurs `null` il faut suffixer le type par un point d'interrogation ?

```
var aUnsafe : String? = "Kotlin"
```

Null safety

- pour autoriser les valeurs `null` il faut suffixer le type par un point d'interrogation ?

```
var aUnsafe : String? = "Kotlin"
```

- mais le compilateur refusera d'accèder à une propriété d'une variable qui peut être nulle!

```
var aUnsafe : String? = "Kotlin"
aUnsage.length
```

→ **Error...**

Null safety

- pour autoriser les valeurs `null` il faut suffixer le type par un point d'interrogation ?

```
var aUnsafe : String? = "Kotlin"
```

- mais le compilateur refusera d'accèder à une propriété d'une variable qui peut être nulle!

```
var aUnsafe : String? = "Kotlin"
aUnsage.length
```

→ **Error...**

- Sauf si on vérifie explicitement que la variable est non nulle:

```
val aUnsafe : String? = "Kotlin"
if (aUnsafe != null) aUnsafe.length else -1
```

Null safety

- Autre solution: utiliser des « **safe call** » `.?`

```
var aUnsafe : String? = "Kotlin"
aUnsafe?.length
```

→ 6

`aUnsafe?.length` renvoie `aUnsafe.length` si `aUnsafe` est non null et renvoie `null` sinon.
(mécanisme standard de Java)

Structures de Contrôle

Conditionnelle « if … else »

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) { return a }
    else { return b }
}
```

Conditionnelle « if … else »

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) { return a }
    else { return b }
}
```

ou plus simplement:

```
fun maxOf(a: Int, b: Int): Int {
    return if (a > b) a else b
}
```

Conditionnelle « if … else »

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) { return a }
    else { return b }
}
```

ou plus simplement:

```
fun maxOf(a: Int, b: Int): Int {
    return if (a > b) a else b
}
```

et encore plus simplement

```
fun maxOf(a: Int, b: Int) = if (a > b) a else b
```

car **la structure `if` est une expression**!

Conditionnelle « when »

Pour simplifier l'imbrication des `if`:

```
when (x) {
    1 -> print("x == 1")
    2 -> print("x == 2")
    else -> {
        print("x est ni 1 ni 2")
    }
}
```

Conditionnelle « when »

Pour simplifier l'imbrication des `if`:

```
when (x) {
    1 -> print("x == 1")
    2 -> print("x == 2")
    else -> {
        print("x est ni 1 ni 2")
    }
}
```

La condition peut se trouver directement dans les branches:

```
when {
    x < 0 -> print("x < 0")
    x > 0 -> print("x > 0")
    else -> {
        print("x == 0")
    }
}
```

Conditionnelle « when »

⚠️ comme pour `if`, l'ordre des branches est important!

```
var x = -1
when {
    x < 0 -> print("x < 0")
    x == -1 -> print("x == -1")
    x > 0 -> print("x > 0")
    else -> {
        print("x == 0")
    }
} // affiche "x < 0"
```

```
var x = -1
when {
    x == -1 -> print("x == -1")
    x < 0 -> print("x < 0")
    x > 0 -> print("x > 0")
    else -> {
        print("x == 0")
    }
} // affiche "x == -1" !
```

Conditionnelle « when »

plutôt que

```
when (x) {
    1 -> print("x == 1")
    2 -> print("x == 2")
    else -> {
        print("pouet pouet")
    }
}
```

on peut aussi écrire:

```
when (x) {
    1, 2 -> print("x == $x")
    else -> {
        print("pouet pouet")
    }
}
```

Conditionnelle « when »

plutôt que

```
when (x) {
    1, 2, 3, 4 -> print("x == $x")
    else -> {
        print("x is not in 1..4")
    }
}
```

on peut aussi écrire:

```
when (x) {
    in 1..4 -> print("x == $x")
    else -> {
        print("x is not in 1..4")
    }
}
```

Boucles « for »

```
for (i in 1..10){
  print("$i ")
}
```

→ 1 2 3 4 5 6 7 8 9 10

```
for (i in 1 until 10){
    print("$i ")
}
```

→ 1 2 3 4 5 6 7 8 9

```
  for (i in 10 downTo 1)
{
    print("$i ")
  }
```

→ 10 9 8 7 6 5 4 3 2 1

```
for ((i,j) in (10 downTo 1).withIndex())
{
    print("$i:$j ")
}
```

→ 0:10 1:9 2:8 3:7 4:6 5:5 6:4 7:3 8:2 9:1

Boucles « while »

```
var i = 1
while (i <= 10) {
    print("$i ")
    i++
}
```

```
var i = 1
do {
    print("$i ")
    i++
} while (i <= 10)
```

→ 1 2 3 4 5 6 7 8 9 10

Les fonctions

Arguments de fonction non mutables

Les arguments d'une fonction sont non mutables!

On **ne peut pas** écrire

```
fun foo(x : Int) : Int {
    x = x+1 // Error: val connot be reassigned
    return x
}
```

Arguments par défaut

- Il est possible de donner des valeurs par défaut aux arguments

```
fun foo(x : Int = 0) : Int {
    return x + 1
}
```

```
print("${foo(2)} ${foo()}")
```

→ `3 1`

Arguments par défaut

- Il est possible de donner des valeurs par défaut aux arguments

```
fun foo(x : Int = 0) : Int {
    return x + 1
}
```

```
print("${foo(2)} ${foo()}")
```

→ `3 1`

- Si une valeur par défaut précède un argument sans, il faut préciser le nom de l'argument:

```
fun plus(x: Int = 0, y: Int) : Int {
    return x + y
}
```

```
print("${plus(1,2)} ${plus(y=1)}")
```

→ `3 1`

Unit

- Si une **fonction ne renvoie rien, son type de retour est `Unit`** (c'est le type qui ne contient qu'une seule valeur)

```
fun foo(x:Int) : Unit {
    println(x)
}
```

Unit

- Si une **fonction ne renvoie rien, son type de retour est `Unit`** (c'est le type qui ne contient qu'une seule valeur)

```
fun foo(x:Int) : Unit {
    println(x)
}
```

- en fait, dans ce cas, `Unit` peut aussi être omis:

```
fun foo(x:Int) {
    println(x)
}
```

mais sa signature reste

```
fun foo(x: Int): Unit
```

Unit

- Si une **fonction ne renvoie rien, son type de retour est `Unit`** (c'est le type qui ne contient qu'une seule valeur)

```
fun foo(x:Int) : Unit {
    println(x)
}
```

- en fait, dans ce cas, `Unit` peut aussi être omis:

```
fun foo(x:Int) {
    println(x)
}
```

mais sa signature reste

```
fun foo(x: Int): Unit
```

- mis à part ce cas particulier, **le type de retour d'une fonction doit toujours être explicitement spécifié** (il ne peut être inféré par le compilateur)

Portée des fonctions

- les fonctions peuvent être créées en dehors des classes

Portée des fonctions

- les fonctions peuvent être créées en dehors des classes
- les fonctions peuvent être déclarées à l'intérieur d'autres fonctions (**fonction locales**)

Lambda expressions (primeur...)

- Dans Kotlin, les fonctions sont des valeurs de première classe (« **first-class citizen** »), i.e. elles peuvent
  - être assignées à des variables,
  - passées en paramètre de fonctions et
  - renvoyées par des fonctions.

```
fun foo(x:Int, f : (Int) -> Int) = println(f(x))
val inc = {x:Int -> x+1} // lambda expression
foo(1, inc)
```

Lambda expressions (primeur...)

- Quand le dernier paramètre est une lambda expression, celle-ci peut-être passée en dehors des parenthèses:

```
foo(1) { x: Int -> x + 1 }
```

Lambda expressions (primeur...)

- Quand le dernier paramètre est une lambda expression, celle-ci peut-être passée en dehors des parenthèses:

```
foo(1) { x: Int -> x + 1 }
```

- Quand l'unique paramètre est une lambda expression, les parenthèses peuvent être omises

```
fun foo(f : (Int) -> Int)) = println(f(0))
foo { x: Int -> x + 1 }
```

Lambda expression d'un seul paramètre

- Il est très courant d'écrire des fonctions d'un seul paramètre. Par exemple:

```
val estZero : (Int) -> Boolean = {x : Int -> x == 0}
```

- `estZero` peut aussi s'écrire:

```
val estZero : (Int) -> Boolean = {it == 0}
```

le compilateur déduit du type de `estZero` qu'il s'agit d'une fonction d'un argument implicitement nommé `it`

Lambda expression d'un seul paramètre

- Il est très courant d'écrire des fonctions d'un seul paramètre. Par exemple:

```
val estZero : (Int) -> Boolean = {x : Int -> x == 0}
```

- `estZero` peut aussi s'écrire:

```
val estZero : (Int) -> Boolean = {it == 0}
```

le compilateur déduit du type de `estZero` qu'il s'agit d'une fonction d'un argument implicitement nommé `it`

- ⚠️ il faut toujours utiliser l'identifiant **`it`**

```
val estZero : (Int) -> Boolean = {x == 0} // Error: Unresolved reference: x
```

Les Collections

Les collections

- 3 types: `List<T>`, `Set<T>`, `Map<K,V>`
- par défaut toutes les collections sont **non mutables**
- variants mutables: `MutableList<T>`, `MutableSet<T>`, `MutableMap<K,V>`

Les listes

- type `List<T>`
- déclaration d'une liste

```
val copains : List<String> = listOf("David", "Andre", "Dominique")
```

Les listes

- type `List<T>`
- déclaration d'une liste

```
val copains : List<String> = listOf("David", "Andre", "Dominique")
```

- accès aux éléments d'une liste (indicés à partir de 0):

```
println(copains.get(0)) // David
```

Les listes

- type `List<T>`
- déclaration d'une liste

```
val copains : List<String> = listOf("David", "Andre", "Dominique")
```

- accès aux éléments d'une liste (indicés à partir de 0):

```
println(copains.get(0)) // David
```

ou bien

```
println(copains[0]) // David
```

Les listes

- type `List<T>`
- déclaration d'une liste

```
val copains : List<String> = listOf("David", "Andre", "Dominique")
```

- accès aux éléments d'une liste (indicés à partir de 0):

```
println(copains.get(0)) // David
```

ou bien

```
println(copains[0]) // David
```

- longueur d'une liste:

```
copains.size // 3
```

Les listes (suite)

- Deux listes sont égales si elles ont des éléments structurellement égaux dans le même ordre et en même nombre (→ sur les collections == teste l'égalité **logique**)

```
val l1 = listOf(5,2,2,4)
val l2 = listOf(5,2,2,4)
println(l1==l2) // true
```

Les listes (suite)

- Deux listes sont égales si elles ont des éléments structurellement égaux dans le même ordre et en même nombre (→ sur les collections == teste l'égalité **logique**)

```
val l1 = listOf(5,2,2,4)
val l2 = listOf(5,2,2,4)
println(l1==l2) // true
```

⚠️ En java, sur les listes, == teste d'égalité **physique**:

```
List<Integer> l1 = List.of(1,2,3);
List<Integer> l2 = List.of(1,2,3);
System.out.println(l1 == l2); // false
```

Les listes (suite)

- Listes mutables

```
val nombres = mutableListOf(1, 2, 3, 4)
nombres.add(5)
nombres.removeAt(1)
numbers[0] = 0
```

Les ensembles

- type `Set` (∼ liste sans ordre et sans duplicats)

Les ensembles

- type `Set` (∼ liste sans ordre et sans duplicats)
- déclaration d'un ensembles

```
val velos : Set<String> = setOf("VTT", "route", "gravel", "tandem")
```

Les ensembles

- type `Set` (∼ liste sans ordre et sans duplicats)
- déclaration d'un ensembles

```
val velos : Set<String> = setOf("VTT", "route", "gravel", "tandem")
```

- deux ensembles sont égaux s'ils ont le même nombre d'éléments et si tout élément d'un ensemble est structurellement égal à un élément de l'autre ensemble:

```
val nombres = setOf(1, 2, 3, 4)
println("Nombre d'éléments: ${nombres.size}")
if (nombres.contains(1)) println("1 est dans l'ensemble")

val autresNombres = setOf(4, 3, 2, 1)
println("Les ensembles sont égaux: ${nombres == autresNombres}") // true
```

Les dictionnaires

- type `Map`
- déclaration d'un dictionnaire

```
val roule : Map<String,String> = mapOf(
    "David" to "route",
    "Andre" to "gravel",
    "Dominique" to "tandem"
)
```

Les dictionnaires

- accès aux éléments d'un dictionnaire

```
println(roule["Dominique"]) // "tandem"
println("Les copains: ${roule.keys}") // Les copains: [David, Andre,
Dominique]
println("Leurs vélos: ${roule.values}") // Leurs vélos: [route, gravel,
tandem]

if ("David" in roule) print("David roule!") // "David roule"
if (roule.containsValue("gravel")) println("Quelqu'un roule en gravel")
```

Les dictionnaires (suite)

- deux dictionnaires sont égaux s'ils contiennent les mêmes paires clé/valeur indépendamment de l'ordre
- dictionnaires mutables

```
val roule = mutableMapOf(
        "David" to "route",
        "Andre" to "VTT",
)

roule.put("Dominique", "tandem")
roule["Andre"] = "gravel"
```

Les tableaux

Les tableaux

- Déclaration d'un tableau avec des valeurs:

```
val tableau = arrayOf(12.0 , 7.0 , 10.5 , 8.2 , 17.8)
```

- ou avec des valeurs `null`:

```
val tableau = arrayOfNulls<Double>(4)
```

⚠️ dans ce cas, il faut donc déclarer explicitement les types des éléments et la taille du tableau

- Taille d'un tableau

```
tableau.size
```